DOMAIN DRIVEN DESIGN USING JAVA

SpotMyDeal

Entrega da 4ª Sprint — API Restful com Spring Boot e Oracle

NOME DA EQUIPE

NextStage

NOME DA SOLUÇÃO

SpotMyDeal

INTEGRANTES

Tayna Jimenes Silva	RM 569337
Gabriel Augusto	RM 573120
Rodrigo Banharelli de Souza	RM 570539
Nycolas Melo Escobar	RM 573052

Sumário

1 Objetivo 3

2 Escopo 3

3 Arquitetura em camadas 4

4 Modelo de Entidade-Relacionamento 5

5 Diagrama de classes 6

6 Endpoints da API 8

7 Regras de negócio 9

8 Tratamento de erros 10

9 Deploy em nuvem 11

10 Integração com o front-end 12

11 Procedimentos para execução 13

12 Repositórios e links 14

1 Objetivo

Desenvolver uma plataforma Java que usa gamificação para incentivar práticas sustentáveis. O sistema conecta usuários a missões ambientais e converte ações positivas em benefícios reais, na forma de cupons em lojas parceiras próximas a quem cumpriu a missão.

Nesta quarta sprint, a solução deixou de ser uma aplicação de console e passou a ser uma **API Restful**, publicada em nuvem e consumida por uma interface web. O domínio continua o mesmo das sprints anteriores; o que mudou foi a forma de acesso e a organização interna do código.

2 Escopo

Perfil do usuário

Cadastro, consulta, alteração e remoção de usuários, com progresso, pontuação acumulada e nível de engajamento calculado a partir dos pontos.

Catálogo de missões

Missões sustentáveis com regra de recompensa e prazo. Cada missão vale uma quantidade de pontos e tem um limite de dias para ser considerada válida.

Geolocalização

Validação de proximidade entre o usuário e as lojas parceiras, com cálculo de distância real sobre coordenadas geográficas e listagem das lojas dentro de um raio informado.

Sistema de gamificação

Registro de conclusão de missão, progressão automática de nível, ranking dos usuários por pontuação e liberação condicionada de benefícios.

Validação de ofertas

Controle do resgate de cupom com base em dois critérios simultâneos: a localização física do usuário no momento do resgate e o histórico recente de atividades sustentáveis.

> **Correção em relação à 3ª Sprint.** O modelo anterior guardava a conclusão da missão como um atributo da própria missão. O efeito era que, quando um usuário concluía uma missão, ela aparecia concluída para todos. Na 4ª Sprint isso virou tabela associativa: a conclusão pertence ao par usuário × missão, não à missão sozinha.

3 Arquitetura em camadas

A requisição percorre sempre o mesmo caminho, e sempre num único sentido. Nenhuma camada pula a seguinte, e nenhuma volta para a anterior.



CAMADA	PACOTE	RESPONSABILIDADE
Model	`model`	As cinco entidades do MER, cada uma com os métodos de negócio que dizem respeito a ela mesma
DAO	`dao`	O SQL de uma tabela cada, com `PreparedStatement` e `try-with-resources`
Conexão	`factory`	`ConnectionFactory` sobre o `DataSource` administrado pelo Spring
BO	`bo`	Regras de negócio e validações; orquestra mais de um DAO quando a regra exige
Resource	`resource`	Os endpoints HTTP; recebe e devolve DTO, e não conhece SQL
DTO	`dto`	Contrato de entrada, com Bean Validation, e contrato de saída
Exceções	`exception`	Exceções de negócio e o `@RestControllerAdvice` que as traduz em status HTTP

Por que a BO existe

A camada de regra de negócio é o que separa o que o banco aceita do que o sistema permite. O Oracle barra a exclusão de um usuário com histórico por causa da chave estrangeira, mas o erro que chega ao cliente é uma falha de integridade, genérica e sem explicação. Com a BO no caminho, a mesma situação vira uma resposta `400` dizendo exatamente o que impede a operação e o que fazer a respeito.

Por que o DTO existe

A entidade e o contrato HTTP mudam por motivos diferentes. Separar os dois evita que o JSON exposto dependa do formato das colunas, e permite recusar na entrada campos que o cliente não deve informar — a pontuação, por exemplo, que só muda por conclusão de missão.

Algoritmos escritos à mão

Três cálculos foram implementados sem recorrer a biblioteca pronta: a distância entre duas coordenadas pela fórmula de Haversine, a ordenação do ranking por Bubble Sort e a validação de formato de e-mail por varredura de caracteres.

4 Modelo de Entidade-Relacionamento

Cinco tabelas, todas com o prefixo `T_SMD_`. As duas em verde são associativas e resolvem relações N:N; as chaves primárias são geradas por *sequence*.

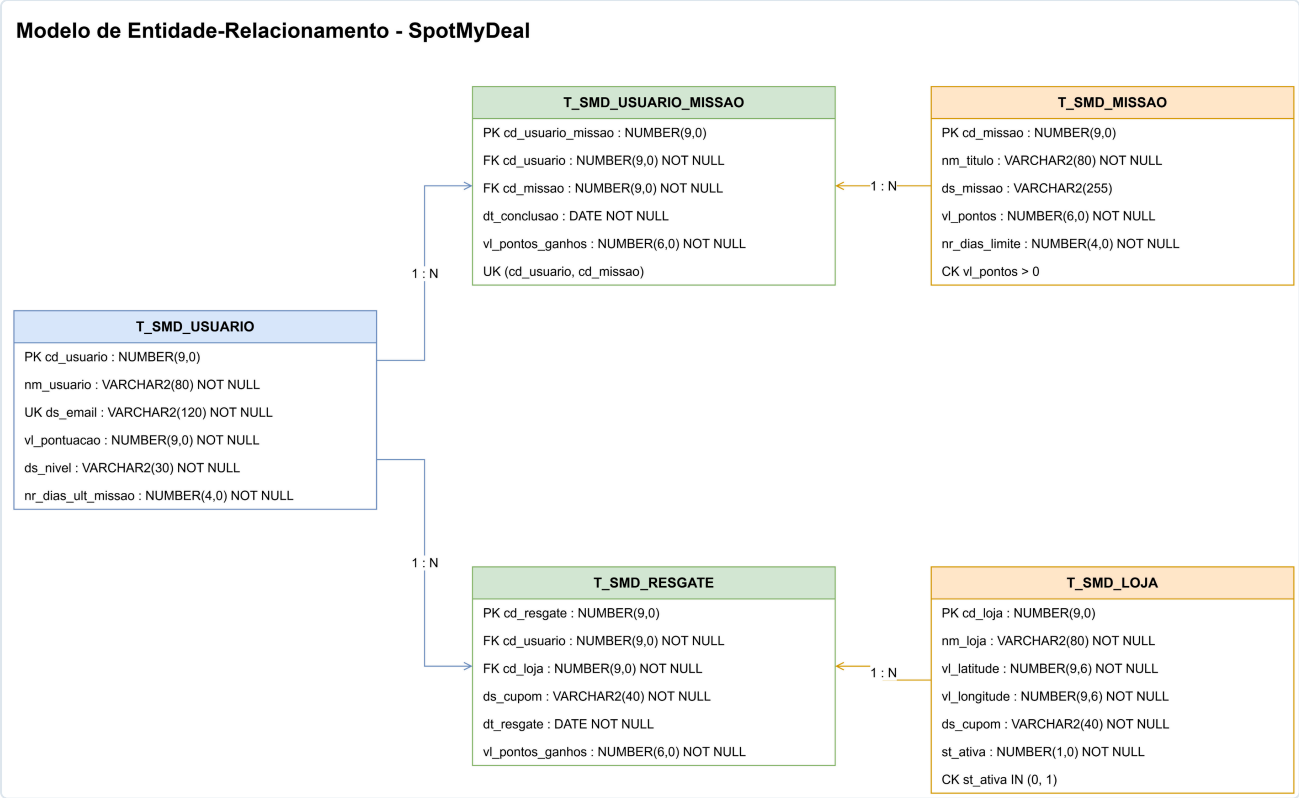


TABELA	O QUE GUARDA
`T_SMD_USUARIO`	Quem participa, com pontuação, nível e dias desde a última missão
`T_SMD_LOJA`	Loja parceira, com coordenadas, cupom e indicador de loja ativa
`T_SMD_MISSAO`	Missão sustentável, com pontos de recompensa e prazo em dias
`T_SMD_USUARIO_MISSAO`	A conclusão: qual usuário concluiu qual missão, quando e por quantos pontos
`T_SMD_RESGATE`	O resgate: qual usuário pegou qual cupom, em qual loja e quando

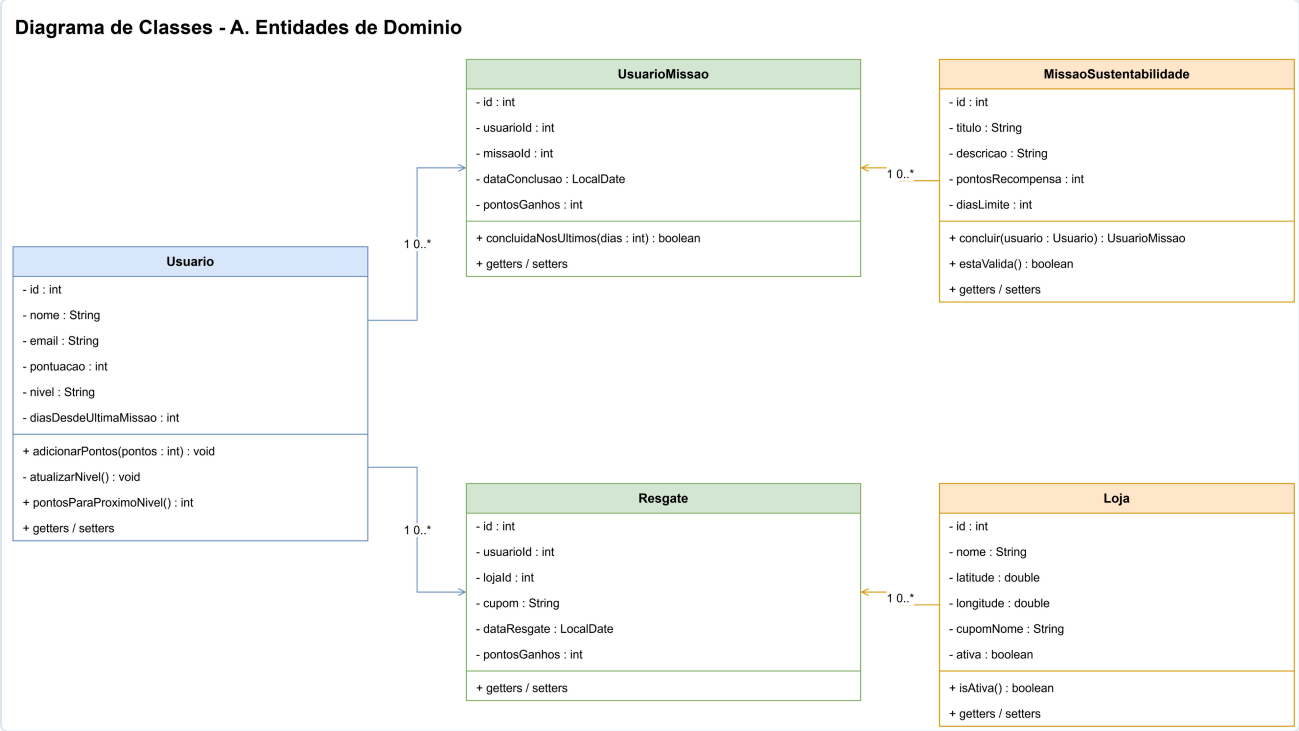
A chave única sobre `(cd_usuario, cd_missao)` é o que garante, no próprio banco, que ninguém conclui a mesma missão duas vezes.

5 Diagrama de classes

A estrutura foi dividida em duas visões, para que cada uma caiba numa leitura.

A. Entidades de domínio

As classes que representam os dados e carregam as regras de negócio de cada entidade.



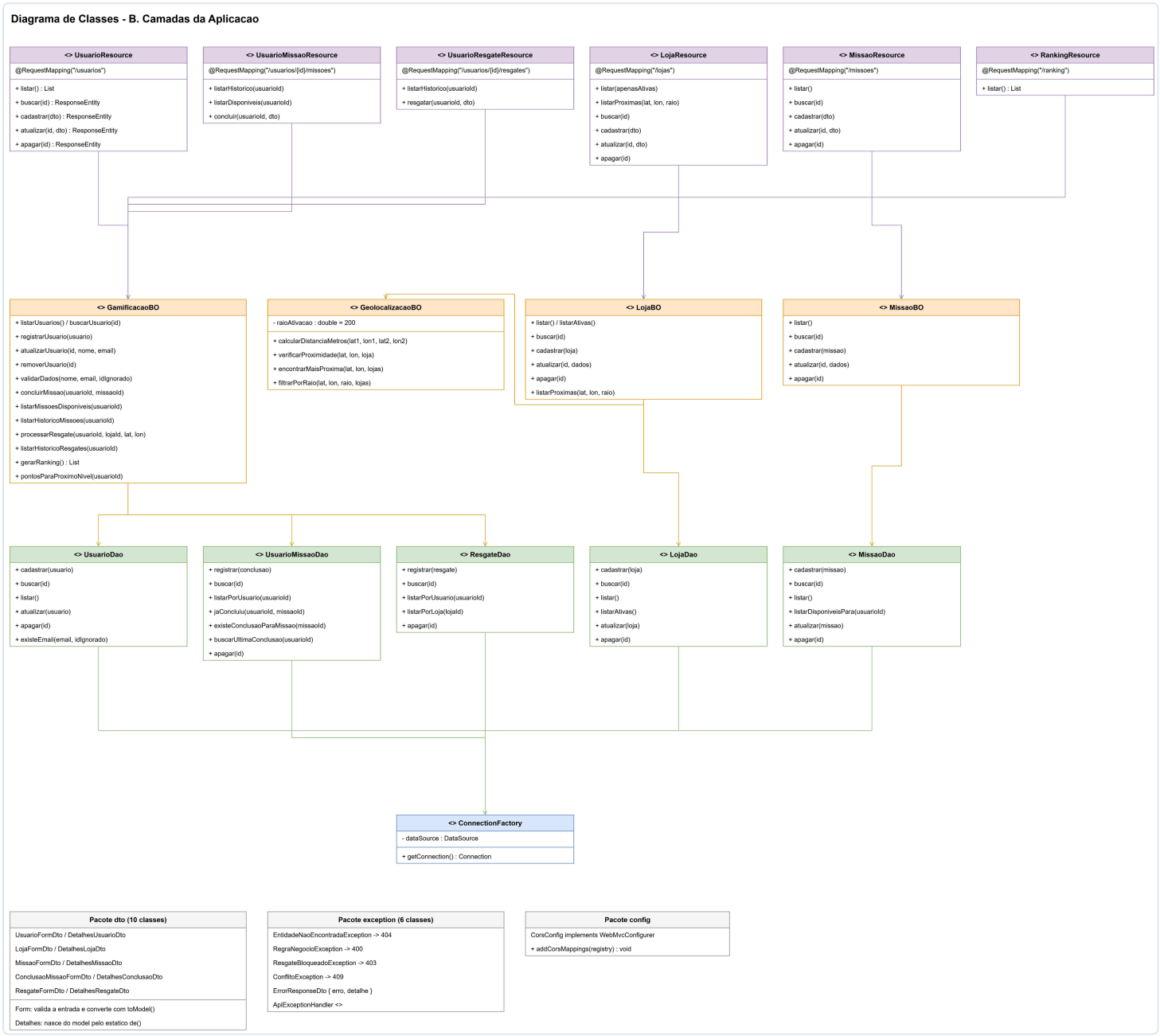
Métodos de negócio de cada entidade

Além dos construtores e acessadores, cada classe carrega os cálculos que dizem respeito a ela mesma. Essa é a diferença entre uma entidade e uma estrutura de dados.

CLASSE	MÉTODO	O QUE RESOLVE
`Usuario`	`adicionarPontos`	Soma os pontos e dispara o recálculo do nível
`Usuario`	`pontosParaProximoNivel`	Quanto falta para subir de faixa
`MissaoSustentabilidade`	`concluir`	Gera a conclusão para um usuário
`MissaoSustentabilidade`	`estaValida`	Verifica se o prazo da missão faz sentido
`UsuarioMissao`	`concluidaNosUltimos`	Se a conclusão está dentro da janela de dias
`Loja`	`isAtiva`	Se a loja pode participar de resgate

B. Camadas da aplicação

Como as classes se organizam em camadas e quem depende de quem. As setas acompanham o sentido da requisição e nunca voltam: cada `Resource` depende apenas da BO correspondente, cada BO depende dos DAO de que precisa, e todos os DAO passam pela mesma `ConnectionFactory` — convergência que torna a troca de banco um problema de uma classe só.



6 Endpoints da API

VERBO	URI	DESCRIÇÃO
GET POST	`/usuarios`	Lista e cadastra usuários
GET PUT DELETE	`/usuarios/{id}`	Busca, atualiza e remove um usuário
GET	`/usuarios/{id}/missoes`	Histórico de missões concluídas
GET	`/usuarios/{id}/missoes/disponiveis`	Missões que o usuário ainda não concluiu
POST	`/usuarios/{id}/missoes`	Registra a conclusão de uma missão
GET	`/usuarios/{id}/resgates`	Histórico de cupons resgatados
POST	`/usuarios/{id}/resgates`	Resgata um cupom numa loja
GET POST	`/lojas`	Lista e cadastra lojas
GET	`/lojas/proximas`	Lojas ativas no raio informado
GET PUT DELETE	`/lojas/{id}`	Busca, atualiza e remove uma loja
GET POST	`/missoes`	Lista e cadastra missões
GET PUT DELETE	`/missoes/{id}`	Busca, atualiza e remove uma missão
GET	`/ranking`	Usuários ordenados por pontuação

Exemplo de resposta

Consulta de um usuário. O nível e os pontos que faltam para o próximo nível são calculados, e não existem como campo de entrada.

```
GET /usuarios/2

{
  "id": 2,
  "nome": "Tayna Jimenes",
  "email": "tayna@soulup.com.br",
  "pontuacao": 1890,
  "nivel": "Lenda Verde",
  "pontosParaProximoNivel": 0
}
```

Busca por proximidade

```
GET /lojas/proximas?lat=-23.559600&lon=-46.658900&raio=500
```

Devolve apenas lojas ativas, ordenadas da mais próxima para a mais distante. Lojas inativas ficam de fora por mais perto que o usuário esteja.

7 Regras de negócio

Liberação do cupom

O resgate exige que **as duas** condições sejam verdadeiras ao mesmo tempo:

- o usuário estar a menos de **200 metros** da loja, pela distância calculada em Haversine;
- ter concluído **alguma missão nos últimos 7 dias**.

Falhando qualquer uma, a resposta é `403` e a mensagem diz qual das duas falhou. Não se trata de erro de permissão nem de autenticação: é regra de negócio, e o texto é escrito para ser mostrado ao usuário final.

Níveis de engajamento

PONTUAÇÃO	NÍVEL
de 0 a 499	Iniciante
de 500 a 799	Ecologista
de 800 a 999	Sustentavel Avancado
1000 ou mais	Lenda Verde

O nível é recalculado sozinho sempre que o usuário ganha pontos, dentro do próprio método `adicionarPontos` da entidade.

Conclusão de missão

A mesma pessoa não conclui a mesma missão duas vezes. A segunda tentativa devolve `409`, e a garantia final está na chave única do banco.

Exclusão com histórico

Usuário com missão concluída ou cupom resgatado não pode ser excluído, assim como loja com resgate e missão com conclusão. Apagar levaria junto o histórico que aponta para o registro. A BO verifica antes e responde `400` com a explicação, em vez de deixar o banco recusar e devolver um erro genérico.

8 Tratamento de erros

Todas as exceções passam por um único `@RestControllerAdvice`, que traduz cada tipo em um status HTTP e devolve sempre o mesmo formato de corpo. Do lado de quem consome, um tratador resolve a tela inteira.

```
{
  "erro": "Resgate bloqueado",
  "detalhe": "Voce ainda nao chegou na area da oferta."
}
```

STATUS	QUANDO ACONTECE
200	Consulta ou alteração concluída
201	Recurso criado
204	Remoção concluída, sem corpo de resposta
400	Campo inválido, id não numérico ou regra de negócio violada
403	Resgate bloqueado por distância ou por falta de missão recente
404	O id informado não existe
409	Missão já concluída por esse usuário
500	Falha de acesso ao banco

Validação de entrada

Os DTO de entrada usam Bean Validation. Quando há mais de um problema no mesmo envio, as mensagens vêm concatenadas, para que o formulário possa apontar todos de uma vez.

```
{
  "erro": "Dados invalidos",
  "detalhe": "E-mail invalido; O nome deve ter entre 3 e 80 caracteres;"
}
```

Mensagem de erro de banco

A mensagem original do Oracle é registrada no log do servidor, mas não vai para a resposta. Erros de banco carregam nomes de tabela, de coluna e de constraint, e isso não deve sair pela API. O cliente recebe uma mensagem genérica.

9 Deploy em nuvem

A aplicação está publicada e respondendo em endereço público.

PLATAFORMA	EMPACOTAMENTO	BANCO
Render	Docker multi-stage	Oracle FIAP

```
https://spotmydeal-sprint4.onrender.com
```

Imagem Docker

O `Dockerfile` tem dois estágios. O primeiro compila com Maven e é descartado inteiro; o segundo recebe apenas o `.jar` e roda sobre uma imagem com JRE 21. O resultado é uma imagem final sem Maven, sem código-fonte e sem as dependências de compilação.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

Credenciais

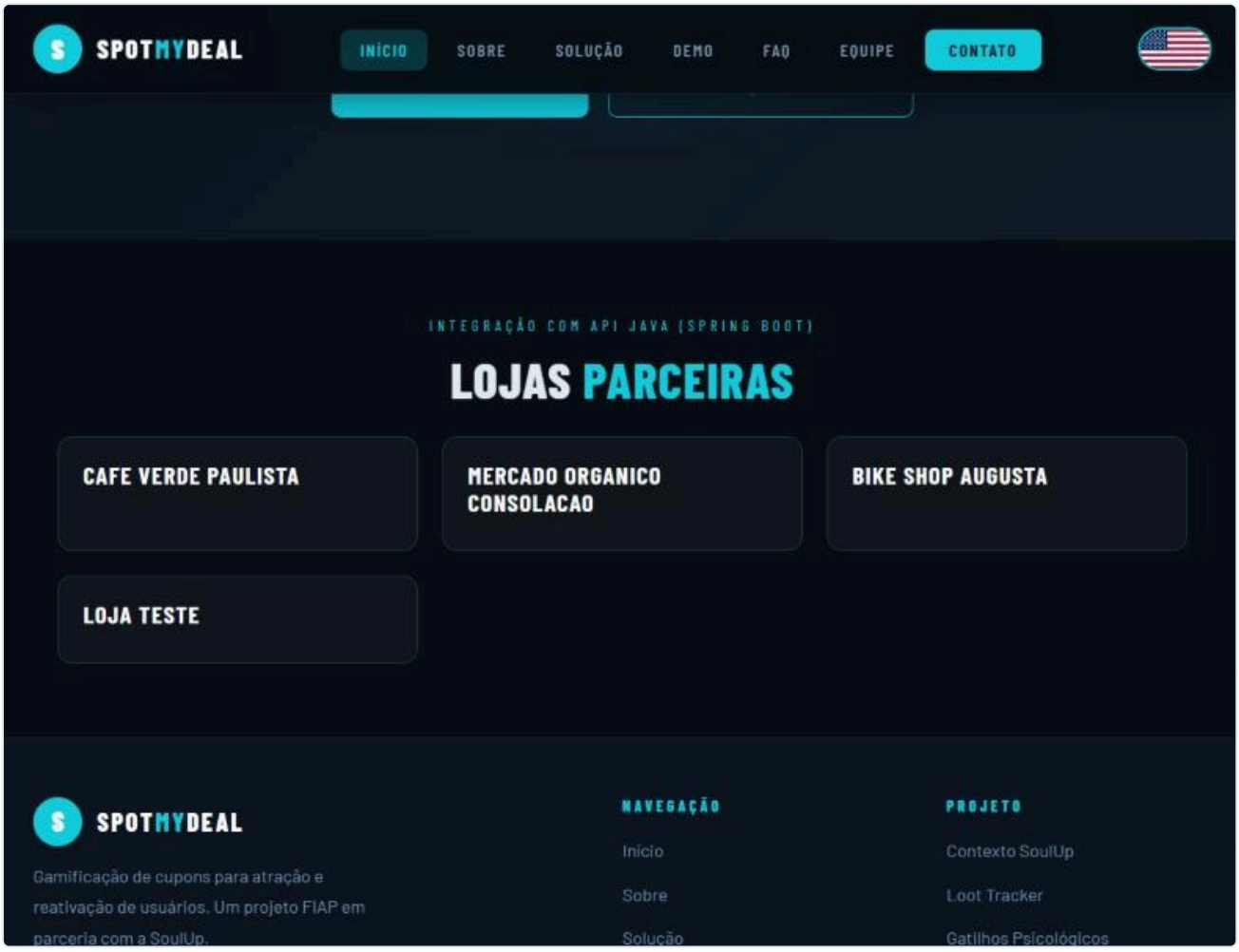
Nenhuma senha está no código. O `application.properties` lê usuário e senha de variável de ambiente, cadastradas no painel do provedor, e a porta vem de `PORT`, que o próprio Render injeta.

```
spring.datasource.url=${DB_URL:jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl}
spring.datasource.username=${DB_USER:rm570539}
spring.datasource.password=${DB_PASS:}
server.port=${PORT:8080}
```

A instância é do plano gratuito e hiberna após 15 minutos sem acesso, de modo que a primeira requisição depois desse intervalo pode levar até 50 segundos até a aplicação responder.

10 Integração com o front-end

A interface web, feita em React e publicada na Vercel, consome esta API. A seção **Lojas Parceiras** da página inicial é montada com o retorno de `GET /lojas`.



Página inicial do SpotMyDeal listando as lojas vindas do Oracle através da API

As quatro lojas exibidas são exatamente as quatro linhas da tabela `T_SMD_LOJA`, incluindo a que foi cadastrada por requisição `POST` durante os testes — o que mostra que a tela reflete o banco, e não uma lista fixa no código.

CORS

Como o front está num domínio diferente do da API, o navegador exigiria autorização explícita para a chamada. A configuração de CORS libera as origens para os métodos GET, POST, PUT, DELETE e OPTIONS; sem ela, a requisição seria barrada antes mesmo de chegar ao servidor.

11 Procedimentos para execução

1. Criar a estrutura do banco

Abrir o `script.sql` no SQL Developer, conectado ao usuário Oracle da FIAP, e executar o arquivo inteiro. Ele cria cinco tabelas, cinco sequences, as chaves estrangeiras e as restrições, e insere os dados de exemplo. Ao final, executa consultas de conferência.

Para recomeçar do zero, os comandos de `drop` estão comentados no topo do arquivo, já na ordem correta: as tabelas filhas primeiro, senão o Oracle recusa a exclusão por causa das chaves estrangeiras.

2. Informar as credenciais

```
set DB_USER=seu_rm
set DB_PASS=sua_senha_do_oracle
```

3. Subir a aplicação

```
mvnw spring-boot:run
```

A API responde em `http://localhost:8080`. Para conferir que subiu, basta chamar `GET /usuarios`: a resposta deve ser `200` com a lista de usuários.

4. Testar os endpoints

O repositório acompanha uma coleção do Insomnia com as requisições prontas, organizadas por assunto e com corpo de exemplo em cada POST e PUT. A importação é feita em **File** → **Import**, e o ambiente já aponta para a URL de produção.

> **Observação sobre a raiz.** O endereço `/` não tem rota mapeada e responde `404`, o que é o comportamento esperado de uma API Restful: não existe página inicial, apenas os recursos. Para verificar que a aplicação está no ar, deve-se chamar `/usuarios`.

Requisitos

ITEM	VERSÃO
Java	21
Maven	3.9.11 — o wrapper acompanha o projeto
Spring Boot	4.1.2
Driver Oracle	ojdbc17

12 Repositórios e links

API Java — esta entrega

ITEM	ENDEREÇO
Código-fonte	`github.com/RodrigoBanharelli/spotmydeal-sprint4`
Aplicação publicada	`spotmydeal-sprint4.onrender.com`
Verificação rápida	`spotmydeal-sprint4.onrender.com/usuarios`

Front-end

ITEM	ENDEREÇO
Código-fonte	`github.com/GRibeiro-developer/Spotmydeal-REACT`
Aplicação publicada	`spotmydeal-react-790tl8nny-gabriel-augusto1.vercel.app`

O que está no repositório da API

CAMINHO	CONTEÚDO
`src/main/java`	As sete camadas, em 39 classes
`script.sql`	Criação das tabelas, sequences, constraints e dados de exemplo
`Dockerfile`	Imagem multi-stage usada no deploy
`README.md`	Arquitetura, endpoints e instruções de execução
`entrega/`	O projeto compactado em `.zip` e esta documentação em PDF, conforme exigido na entrega
`docs/`	Fontes dos diagramas usados nesta documentação

Equipe NextStage. Tayna Jimenes Silva (RM 569337), Gabriel Augusto (RM 573120), Rodrigo Banharelli de Souza (RM 570539) e Nycolas Melo Escobar (RM 573052) — 1TDSPW, FIAP, 2º semestre de 2026.